MANTRA BIOTIME SDK WINDOWS - .NET

Ver.: 1.0.0.0

Mantra Softech India Pvt. Ltd.

Version	1.0.0.0
Release Date	1st March, 2020
Author	Mr. Jatinkumar Pansuriya
SDK Support	jatin.pansuriya@mantratec.com +91-96-627-51-663

NOTE: Do not share with your clients. it is only for development support.

About:

The document provides the functional and implementation information to work with Mantra Attendance Device (Fingerprint, Face, Card, Password). By using this SDK, you can get Biometric & Demographic data, attendance logs and access control data from BioTimeSDK. This SDK provided facility to extract different-different interface like –

- Biometric Data
- Demographic Data
- Attendance Log Data
- Timezone Access Control Data

Dependency:

1. Microsoft .Net Framework 2.0
2. MantraBioTimeDLL

Contents:

#	Functions/Events
1	SDK Description
2	ConnectTCP
3	DeviceInfo
4	CapacityInfo
5	TimePeriodLogData
6	ReadNewLogData
7	ReadAllLogData
8	DeleteAllLogData
9	GetUserInfo
10	SetUserInfo
11	GetUserValidDate
12	SetUserValidDate
13	GetUserVerifyStyle
14	SetUserVerifyStyle
15	GetUserPhoto
16	SetUserPhoto
17	DelUserPhoto
18	StartEnrollFP

19	StartEnrollFace
20	StartEnrollCard
21	StopEnroll
22	GetUserEnrollData
23	SetUserEnrollData
24	GetAllUserEnrollData
25	SetAllUserEnrollData
26	DeleteEnrollData
27	DeleteAllEnrollData
28	GetTZone
29	SetTZone
30	GetUserTZone
31	SetUserTZone
32	GetDeviceTime
33	SetDeviceTime
34	SetThisDeviceTime
35	RestartDevice
36	PowerOffDevice
37	ListenerToDevice
38	DisConnectTCP

Note :

DeviceType	Select Device
1	Morx-Enterprise Device
2	mBio/mFace Series Device

Flag:

Return	Operations
0	Function execution success
Error Code	Function execution failure

Terminal:

1. **ConnectTCP (DeviceType)**
   Return: Integer

```
If(MantraBioTime.ConnectTCP(DeviceID, DeviceIP, DevicePort, DevicePassword, DeviceType) ==
0)
{
      Message("Device Connected);
}
else
{
      Message("Device not connected”);
}
```

2. **DeviceInfo (DeviceType)**
   Return: Integer

   getDeviceInfo()

```
MantraBT_DeviceInfo.FirmwareVersion;
MantraBT_DeviceInfo.Mac;
MantraBT_DeviceInfo.SerialNumber;
MantraBT_DeviceInfo.PlatForm;
MantraBT_DeviceInfo.DeviceName;
MantraBT_DeviceInfo.FPAlg;
MantraBT_DeviceInfo.FaceAlg;
MantraBT_DeviceInfo.Manufacturer;
MantraBT_DeviceInfo.ManufactureTime;
```

3. **CapacityInfo (DeviceType)**

   getCapacityInfo()

```
MantraBT_CapacityInfo.AdminCnt;
MantraBT_CapacityInfo.UserCount;
MantraBT_CapacityInfo.FpCnt;
MantraBT_CapacityInfo.recordCnt;
MantraBT_CapacityInfo.PwdCnt;
MantraBT_CapacityInfo.CardCnt;
MantraBT_CapacityInfo.OpLogCnt;
MantraBT_CapacityInfo.FaceCnt;
```

LogData:

VerifyType

DeviceType : 1	DeviceType : 2
"Group Verify" -> 0 "FP/PW/RF" -> 128 "FP" -> 129 "PIN" -> 130 "PW" -> 131 "RF" -> 132 "FP/PW" -> 133 "FP/RF" -> 134 "PW/RF" -> 135 "PIN&FP" -> 136 "FP&PW" -> 137 "FP&RF" -> 138 "PW&RF" -> 139 "FP&PW&RF" -> 140 "PIN&FP&PW" -> 141 "FP&RF/PIN" -> 142	"FP" -> 1 "PASSWORD" -> 2 "CARD" -> 3 "FP+CARD" -> 4 "FP+PWD" -> 5 "FP+PASSWORD" -> 5 "CARD+PWD" -> 6 "CARD+PASSWORD" -> 6 "FP+CARD+PWD" -> 7 "FP+CARD+PASSWORD" -> 7 "HAND LOCK" -> 10 "PROG LOCK" -> 11 "PROG OPEN" -> 12 "PROG CLOSE" -> 13 "AUTO RECOVER" -> 14 "LOCK OVER" -> 20 "ILLEGAL OPEN" -> 21 "DURESS ALARM" -> 22 "TAMPER DETECT" -> 23 "FACE" -> 30 "FACE+CARD" -> 31 "FACE+PWD" -> 32 "FACE+Password" -> 32 "FACE+CARD+PWD" -> 33 "FACE+CARD+Password" -> 33 "FACE+FP" -> 34

VerifyMode

DeviceType : 1	DeviceType : 2
"Check-In Default" -> 0 "Check-Out" -> 1 "Break-Out" -> 2 "Break-In" -> 3 "OT-In" -> 4 "OverTime-In" -> 4 "OverTime-Out" -> 5 "OT-Out" -> 5	"_DUTYON" -> 0 "DUTY ON" -> 0 "_DUTYOFF" -> 1 "DUTY OFF" -> 1 "_OVERON" -> 2 "OVERTIME" ON -> 2 "_OVEROFF" -> 3 "OVERTIME OFF" -> 3 "_GOIN" -> 4 "GO IN" -> 4 "RETURN" -> 4 "_GOOUT" -> 5 "GO OUT" -> 5 "NONE" -> 0 "IN" -> 1 "OUT" -> 2

Note: Card/Badge both are same.

1. **TimePeriodLogData()**
   Return: Integer

   Get AttnLog Data Between To Date

```
int _flgVal = MantraBioTime.TimePeriodLogData(DeviceType,
dtpFromDate.Value.Date.ToString("yyyy-MM-dd"), dtpToDate.Value.Date.ToString("yyyy-MM-dd"),
ref Dt);
if (_flgVal == 0)
{
      Message ("Successfully");
      //Dt Filled
}
else
{
     Message(MantraBioTimeException.MsgCode(_flgVal));
}
```

2. **ReadNewLogData()**
   Return: Integer

   Get Unread AttnLog Data

```
int _flgVal = MantraBioTime.ReadNewLogData(DeviceType, ref Dt);
if (_flgVal != 0)
{
       Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      //Dt Filled
}
```

3. **ReadAllLogData()**
   Return: Integer

   Get All AttnLog Data(1 To last record.)

```
int _flgVal = MantraBioTime.ReadAllLogData(DeviceType, ref Dt);
if (_flgVal != 0)
{
       Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      //Dt Filled
}
```

4. **DeleteAllLogData()**
   Return: Integer

   Delete All AttnLog Data(1 To last record.)

```
int  _flgVal = MantraBioTime.DeleteAllLogData(DeviceType);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message ("Successfully");
}
```

UserData:

User Info :

1. **GetUserInfo()**
   Return: Integer

   Get UserInfo In Device (List->MachineNo,UserId,UserName,UserPassword,UserCard,Privilege,UserEnable)

```
int _flgVal = MantraBioTime.GetUserInfo(DeviceType, UserID, ref List);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      //List Filled
}
```

2. **SetUserInfo()**
   Return: Integer

   Set UserInfo In Device (List->UserId,UserName,UserPassword,UserCard,Privilege,UserEnable)

```
int _flgVal = MantraBioTime.SetUserInfo(DeviceType, UserID, Name, Password, Cardnumber,
Privilege, true);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

3. **GetUserValidDate()**
Return: Integer

Get UserValidDate In Device FromDate, ToDate-DateFormat("yyyy-MM-dd")

```
int _flgVal = MantraBioTime.GetUserValidDate(DeviceType, UserID, ref FromDate, ref ToDate);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      // Filled
      // FromDate
      // ToDate
      Message("Successfully");
}
```

4. **SetUserValidDate()**
Return: Integer

Get UserValidDate In Device (FromDate, ToDate-DateFormat"yyyy-MM-dd")

```
int _flgVal = MantraBioTime.SetUserValidDate(DeviceType, UserID, FromDate, ToDate);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

5. **GetUserVerifyStyle()**
Return: Integer

Get UserVerifyStyle In Device (like "FP" Only)

```
int VerifyStyle = 0;
int _flgVal = MantraBioTime.GetUserVerifyStyle(DeviceType, UserID, ref VerifyStyle);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      //VerifyStyle Filled.
      Message("Successfully");
}
```

6. **SetUserVerifyStyle()**
Return: Integer

Set UserVerifyStyle In Device (like "FP")

```
String VerifyStyle = "FP";
int _flgVal = MantraBioTime.SetUserVerifyStyle(DeviceType, UserID, VerifyStyle);
if (_flgVal != 0)

{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

Others :

1. **GetUserPhoto()**
Return: Integer

```
int _flgVal = MantraBioTime.GetUserPhoto(DeviceType, UserID, PhotoPath, ref imgByte);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
     // Filled Byte array
      Message("Successfully");
}
```

2. **SetUserPhoto()**
Return: Integer

```
int _flgVal = MantraBioTime.SetUserPhoto(DeviceType, UserId, PhotoPath)
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

3. **DelUserPhoto()**
   Return: Integer

```
int _flgVal = MantraBioTime.DelUserPhoto(DeviceType, UserId);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

4. **StartEnrollFP()**
   Return: Integer

```
int _flgVal = MantraBioTime.StartEnrollFP(DeviceType, UserID, FinIndex);
if (_flgVal != 0)

{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

5. **StartEnrollFace()**
   Return: Integer

```
int _flgVal = MantraBioTime.StartEnrollFace(DeviceType, UserID);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

6. **StartEnrollCard()**
   Return: Integer

```
int _flgVal = MantraBioTime.StartEnrollCard(DeviceType, UserID);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

7. **StartEnrollCard()**
Return: Integer

```
int _flgVal = MantraBioTime.StopEnroll(DeviceType);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

User Enrolled Data :

Common for all

```
public class DemographicsList
    {
        public int MachineNo { get; set; }
        public int DeviceType { get; set; }
        public string UserId { get; set; }
        public string UserName { get; set; }
        public string pwd { get; set; }
        public string Card { get; set; }
        public int Privilege { get; set; }
        public bool IsEnabled { get; set; }
    }


public class BiometricsList
    {
        public int MachineNo { get; set; }
        public int DeviceType { get; set; }
        public string UserId { get; set; }
        public int InputNo { get; set; }
        public Byte[] Template { get; set; }
    }
    List<DemographicsList> DList = new List<DemographicsList>();
    List<BiometricsList> BList = new List<BiometricsList>();
```

1. **GetUserEnrollData()**
Return: Integer

```
int _flgVal = MantraBioTime.GetUserEnrollData(DeviceType, UserID, ref DList, ref BList);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      // DList → Demographic Data
      // BList -> → Biometric Data
      Message("Successfully");
```

```
}
```

2. **SetUserEnrollData()**
   Return: Integer

```
Table Fields :-
tblData =>"[Index], MachineNo, DeviceType, UserId, UserName, pwd,
           [Card], Privilege, IsEnabled, InputNo, Template"

int _flgVal = MantraBioTime.SetUserEnrollData(DeviceType, UserID, tblData);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

3. **GetAllUserEnrollData()**
   Return: Integer

```
int _flgVal = MantraBioTime.GetAllUserEnrollData(DeviceType, UserID, ref DList, ref BList);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

4. **SetAllUserEnrollData()**
   Return: Boolean

```
int _flgVal = MantraBioTime.SetAllUserEnrollData(DeviceType, UserID, tblData);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

5. **DeleteEnrollData()**
Return: Integer

```
int _flgVal = MantraBioTime.DeleteEnrollData(DeviceType, UserID);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

6. **DeleteAllEnrollData()**
Return: Integer

```
int _flgVal = MantraBioTime.DeleteAllEnrollData(DeviceType, UserID);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

Access TimeZone :

1. **GetTZone()**
Return: Integer

```
List<KeyValuePair<string, string>> _TZ = new List<KeyValuePair<string,string>>();
int _flgVal = MantraBioTime.GetTZone(DeviceType, TimeZoneID, ref _TZ);
if (_flgVal != 0)

{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

2. **SetTZone()**
Return: Integer

Value Format "00:00[HH:mm]"

```
int _flgVal = MantraBioTime.SetTZone(DeviceType, TimeZoneID, dtSUN.Value, dtTOSUN.Value,
dtMON.Value, dtTOMON.Value, dtTUE.Value, dtTOTUE.Value, dtWEN.Value, dtTOWEN.Value,
dtTHU.Value, dtTOTHU.Value, dtFRI.Value, dtTOFRI.Value, dtSTA.Value, dtTOSTA.Value);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
```

```
}
else
{
      Message("Successfully");
}
```

User Access TimeZone :

1. **GetUserTZone()**
   Return: Integer

```
string idx1 = ""; string idx2 = ""; string idx3 = "";
int _flgVal = MantraBioTime.GetUserTZone(DeviceType, UserAccessTimeZoneID, ref idx1, ref
idx2, ref idx3);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

2. **SetUserTZone()**
   Return: Integer

   Idx range "[1-50]"

```
int _flgVal = MantraBioTime.SetUserTZone(DeviceType, UserAccessTimeZoneID, idx1, idx2, idx3);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

Others :

1. **GetDeviceTime()**
   Return: Integer

```
string GetTime=""; Format("yyyy-MM-dd HH:mm:ss")
int _flgVal = MantraBioTime.GetDeviceTime(clsGlobal.DeviceType, ref GetTime);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

2. **SetDeviceTime ()**
Return: Integer

Your System Time Set in Device.

```
int _flgVal = MantraBioTime.SetDeviceTime(DeviceType);
if (_flgVal != 0)

{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

3. **SetThisDeviceTime ()**
Return: Integer

Your Manual Time Set in Device.

```
int _flgVal = MantraBioTime.SetThisDeviceTime(DeviceType, dtDeviceTime.Value);
if (_flgVal != 0)

{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

4. **RestartDevice ()**
Return: Integer

Restart Device.

```
int _flgVal = MantraBioTime.RestartDevice(DeviceType);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

5. **PowerOffDevice ()**
Return: Integer

Restart Device.

```
int _flgVal = MantraBioTime.PowerOffDevice(DeviceType);
if (_flgVal != 0)
{
      Message(MantraBioTimeException.MsgCode(_flgVal));
}
else
{
      Message("Successfully");
}
```

ErrorCode

Code	Text	Description
0	BioTimeSDK_E_SUCCESS	
-1	BioTimeSDK_E_FAIL	*Operation Fail.
-2	BioTimeSDK_E_INVALIDDATATYPE	Invalid DataType.
-1002	BioTimeSDK_E_INVALIDPARAM	Invalid Parameter.
-1003	BioTimeSDK_E_INVALIDPASS	Invalid Password.
-1004	BioTimeSDK_E_NOTSUPPORTED	May be Device Not Supported.
-1005	BIOTIMESDK_E_FUNCTIONNOTSUPPORTED	May Function Not Supported.
-1006	BIOTIMESDK_E_NUMBERONLY	Number Only.
-1007	BIOTIMESDK_E_USERNOTFOUND	User Not Found.
-1008	BioTimeSDK_E_PHOTONOTFOUND	PHOTO Not Found.
-1009	BIOTIMESDK_E_INVALIDFINGER	Invalid Finger Index.
-1010	BIOTIMESDK_E_ENROLLDATANOTFOUND	EnrollData Not Found.
-2000	BIOTIMESDK_E_ATTENDATANOTFOUND	AttnData Not Found.
-2001	BioTimeSDK_E_UNHANDLE	Exception Unhandle.
		Unknown

-x-x-x-x-x-x-x-x-x-x-x-x-x-x-x-x-x-